



FEU INSTITUTE OF TECHNOLOGY

College of Computer Studies

CS0019
(Modeling and Simulation)

ASSESSMENT

1

Introduction to MATLAB

Student Name:	Dawn Andrei Pamesa
Section:	TS31
Professor:	Sir Abraham Magpantay



FEU INSTITUTE OF TECHNOLOGY

I. Objectives

At the end of this Technical Assessment, the student should be able to:

- Create and manipulate MATLAB vectors and matrices.
- Apply indexing, built-in functions, and element-wise operations.
- Analyze data using MATLAB.
- Use AI to review and improve MATLAB code.
- Verify AI-generated recommendations.

II. Materials

- MATLAB or MATLAB Online
- Generative AI tool - openAI
- Introduction to MATLAB module

III. Problem Description

A company records the daily production of three products over five working days. Develop a MATLAB program that analyzes the production data and simulates a proposed production adjustment.

Use the following data:

```
production = [  
120 95 80;  
135 100 76;  
128 110 85;  
140 105 90;  
150 115 88  
];  
  
sellingPrice = [50 65 80];  
productionCost = [30 40 55];  
adjustmentFactor = [1.10 1.05 0.97];
```

Each row represents a working day, while each column represents Products A, B, and C.

IV. Instruction

Create a MATLAB script that:



FEU INSTITUTE OF TECHNOLOGY

1. Determines the size and number of elements in the production matrix.
2. Calculates the daily and weekly production totals.
3. Calculates the weekly revenue, cost, and profit.
4. Applies the adjustment factors to simulate a new production plan.
5. Compares the original and simulated weekly profits.
6. Displays a recommendation based on the results.
7. Uses AI to review the code for errors.
8. Documents one AI recommendation and explains how it was tested in MATLAB.

VI. Output Requirement

- Attach screenshots showing the full output of each program.
- MATLAB .m or .mlx file
- Screenshot of the successful program output
- Original and simulated production results
- Original and simulated weekly profits
- Brief interpretation and recommendation
- Copy of the AI prompt and relevant response
- Explanation of how one AI suggestion was verified
- Answers to the five questions

VII. Question and Answer

1. What does MATLAB stand for?
MATLAB stands for MATrix LABoratory. The name reflects its original purpose, which was to give engineers and scientists direct access to matrix computation without writing programs in a lower-level language. This is why the matrix is the fundamental data type in MATLAB, and even a single number is stored as a 1 by 1 matrix. In this program, the size command confirms that the production data is held as a 5 by 3 matrix.
2. What do the rows and columns of the production matrix represent?
Each row represents one working day and each column represents one product. The matrix is 5 by 3, so there are five working days and three products, giving 15 elements in total.

This layout determines how the totals are computed. Summing across the columns with `sum(production, 2)` collapses each row and gives the total production for each day. Summing down the rows with `sum(production, 1)` collapses each column and gives the weekly production for each product. For example, `production(3,2)` holds 110, which is the output of Product B on Day 3.
3. What is the difference between `*` and `.*` in MATLAB?
The asterisk performs matrix multiplication, which follows the rules of linear algebra. In an expression `A * B`, the number of columns in A must equal the number of rows in B, and each entry of the result is a sum of products, so values are mixed together.

The dot asterisk performs element-wise multiplication. Each element of one operand is multiplied by the element in the matching position of the other, with no summing, so the result keeps the same shape as the inputs.

This program requires element-wise multiplication. In the statement `simProduction =`



FEU INSTITUTE OF TECHNOLOGY

`round(production .* adjustmentFactor)`, each product column must be scaled by its own factor, so Product A is multiplied by 1.10, Product B by 1.05, and Product C by 0.97. Writing `production * adjustmentFactor` instead would attempt matrix multiplication of a 5 by 3 matrix with a 1 by 3 vector, and since the inner dimensions of 3 and 1 do not agree, MATLAB would stop with a dimension mismatch error.

4. Why is `round()` used in the simulation?

The `round` function is used because production is counted in whole physical units. Multiplying the recorded output by the adjustment factors produces fractional values that cannot exist on a factory floor. On Day 1, Product C gives 80 multiplied by 0.97, which is 77.6, and a company cannot manufacture 77.6 units, so the value is rounded to 78 to keep the simulated plan physically meaningful.

Rounding also keeps the financial results consistent, because revenue and cost are calculated from unit counts. If the fractional values were carried through, the reported profit would describe a production plan that could never actually be executed.

One detail is worth noting. Two adjusted values in this data set land exactly on a half unit, namely 135 multiplied by 1.10, which is 148.5, and 110 multiplied by 1.05, which is 115.5. MATLAB rounds a value ending in exactly one half away from zero, so these become 149 and 116. Languages that round half to the nearest even number would give 148 for the first case, which would change the weekly totals.

Paste your MATLAB code here:



FEU INSTITUTE OF TECHNOLOGY

File: PAMESA_TechnicalAssessment1.m

```
%% CS0019 - Modeling and Simulation
% Technical Assessment 1: Introduction to MATLAB
% Name      : Dawn Andrei Pamesa
% Section   : TS31

clc; clear;

%% Given data
production = [
    120 95 80;
    135 100 76;
    128 110 85;
    140 105 90;
    150 115 88
];

sellingPrice    = [50 65 80];
productionCost  = [30 40 55];
adjustmentFactor = [1.10 1.05 0.97];

productNames = ["Product A", "Product B", "Product C"];

%% 1. Size and number of elements of the production matrix
[numDays, numProducts] = size(production);
totalElements = numel(production);

% Input validation, adopted from the AI review. Each per-product vector must
% hold exactly one value per column of the production matrix.
assert(numel(sellingPrice) == numProducts, 'sellingPrice must contain one value per product.');
```

```
assert(numel(productionCost) == numProducts, 'productionCost must contain one value per product.');
```

```
assert(numel(adjustmentFactor) == numProducts, 'There must be exactly one adjustment factor per product.');
```

```
assert(numel(productNames) == numProducts, 'productNames must contain one name per product.');
```

```
fprintf('=== 1. MATRIX PROPERTIES ===\n');
```

```
fprintf('Size of production matrix : %d rows x %d columns\n', numDays, numProducts);
```

```
fprintf('Number of working days      : %d\n', numDays);
```

```
fprintf('Number of products           : %d\n', numProducts);
```

```
fprintf('Total number of elements     : %d\n\n', totalElements);
```

```
%% 2. Daily and weekly production totals
dailyTotals = sum(production, 2); % sum across the columns, one total per day
productTotals = sum(production, 1); % sum down the rows, one total per product
weeklyTotal = sum(production, 'all');
```

```
fprintf('=== 2. PRODUCTION TOTALS ===\n');
```

```
for d = 1:numDays
    fprintf('Day %d total production : %d units\n', d, dailyTotals(d));
end
fprintf('\n');
```

```
for p = 1:numProducts
    fprintf('%s weekly production: %d units\n', productNames(p), productTotals(p));
end
fprintf('Total weekly production : %d units\n\n', weeklyTotal);
```

```
%% 3. Weekly revenue, cost, and profit
revenuePerProduct = productTotals .* sellingPrice; % element-wise, per product
costPerProduct = productTotals .* productionCost;
profitPerProduct = revenuePerProduct - costPerProduct;
```

```
weeklyRevenue = sum(revenuePerProduct);
weeklyCost = sum(costPerProduct);
weeklyProfit = weeklyRevenue - weeklyCost;
```

```
fprintf('=== 3. ORIGINAL WEEKLY FINANCIALS ===\n');
```

```
fprintf('%-12s %12s %12s %12s\n', 'Product', 'Revenue', 'Cost', 'Profit');
```

```
for p = 1:numProducts
    fprintf('%-12s %12.2f %12.2f %12.2f\n', productNames(p), ...
        revenuePerProduct(p), costPerProduct(p), profitPerProduct(p));
end
fprintf('%-12s %12.2f %12.2f %12.2f\n', 'TOTAL', ...
```



FEU INSTITUTE OF TECHNOLOGY

```
weeklyRevenue, weeklyCost, weeklyProfit);

%% 4. Apply the adjustment factors to simulate a new production plan
simProduction = round(production .* adjustmentFactor);

simProductTotals = sum(simProduction, 1);
simDailyTotals   = sum(simProduction, 2);
simWeeklyTotal   = sum(simProduction, 'all');

simRevenuePerProduct = simProductTotals .* sellingPrice;
simCostPerProduct     = simProductTotals .* productionCost;
simProfitPerProduct   = simRevenuePerProduct - simCostPerProduct;

simWeeklyRevenue = sum(simRevenuePerProduct);
simWeeklyCost    = sum(simCostPerProduct);
simWeeklyProfit  = simWeeklyRevenue - simWeeklyCost;

fprintf('=== 4. SIMULATED PRODUCTION PLAN ===\n');
fprintf('Adjustment factors applied: A = %.2f, B = %.2f, C = %.2f\n\n', ...
    adjustmentFactor(1), adjustmentFactor(2), adjustmentFactor(3));
fprintf('%-8s %12s %12s %12s %12s\n', 'Day', 'Product A', 'Product B', 'Product C', 'Day Total');
for d = 1:numDays
    fprintf('Day %-4d %12d %12d %12d %12d\n', d, ...
        simProduction(d,1), simProduction(d,2), simProduction(d,3), simDailyTotals(d));
end
fprintf('%-8s %12d %12d %12d %12d\n\n', 'TOTAL', ...
    simProductTotals(1), simProductTotals(2), simProductTotals(3), simWeeklyTotal);

fprintf('=== SIMULATED WEEKLY FINANCIALS ===\n');
fprintf('%-12s %12s %12s %12s\n', 'Product', 'Revenue', 'Cost', 'Profit');
for p = 1:numProducts
    fprintf('%-12s %12.2f %12.2f %12.2f\n', productNames(p), ...
        simRevenuePerProduct(p), simCostPerProduct(p), simProfitPerProduct(p));
end
fprintf('%-12s %12.2f %12.2f %12.2f\n\n', 'TOTAL', ...
    simWeeklyRevenue, simWeeklyCost, simWeeklyProfit);

%% 5. Compare the original and simulated weekly profits
profitDifference = simWeeklyProfit - weeklyProfit;
percentChange    = (profitDifference / weeklyProfit) * 100;

fprintf('=== 5. PROFIT COMPARISON ===\n');
fprintf('Original weekly profit      : %.2f\n', weeklyProfit);
fprintf('Simulated weekly profit       : %.2f\n', simWeeklyProfit);
fprintf('Difference                     : %.2f\n', profitDifference);
fprintf('Percent change                 : %.2f%%\n', percentChange);

%% 6. Recommendation based on the results
fprintf('=== 6. RECOMMENDATION ===\n');
if profitDifference > 0
    fprintf(['ADOPT the proposed production adjustment.\n' ...
        'It increases weekly profit by %.2f (%.2f%%).\n'], ...
        profitDifference, percentChange);
elseif profitDifference < 0
    fprintf(['REJECT the proposed production adjustment.\n' ...
        'It decreases weekly profit by %.2f (%.2f%%).\n'], ...
        abs(profitDifference), abs(percentChange));
else
    fprintf(['The adjustment is FINANCIALLY NEUTRAL.\n' ...
        'Weekly profit is unchanged, so the decision should rest on\n' ...
        'other factors such as demand, storage, or material supply.\n']);
end

[bestProfit, bestIdx] = max(simProfitPerProduct);
fprintf('Highest contributor under the new plan: %s (%.2f profit).\n', ...
    productNames(bestIdx), bestProfit);
```



Paste the snippet of your outputs here:

```
NAME: PAMESA - AI Declaration ...
PAMESA_hwk1.docx
PAMESA_hwk1.pdf
PAMESA_TechnicalAssessme...

Workspace
Name Value Size Class
adjustmentfa... [1.1000,1.0500,...] 1x3 double
bestids 1 1x1 double
bestProfit 14820 1x1 double
costPerProduct [20190,21000,21... 1x3 double
d 5 1x1 double
dailyTotals [295,311,323,335... 5x1 double
numDays 5 1x1 double
numProducts 3 1x1 double
p 3 1x1 double
percentChange 4.6141 1x1 double
production 5x3 double
productionC_ [30,40,55] 1x3 double
productNames 1x3 string
productTotals [873,525,419] 1x3 double
profitDifferen... 1710 1x1 double
profitPerProd... [13460,13125,10... 1x3 double

PAMESA_TechnicalAssessment.m
1 % CS0019 - Modeling and Simulation
2 % Technical Assessment 1: Introduction to MATLAB
3 % Name : Dawn Andrei Pamesa
4 % Section : TS31
5
6
7
8 clc; clear;
9
10 % Given data
11 production = [
12 120 95 80;
13 135 100 76;
14 120 110 85;
15 140 105 90;
16 150 115 88
17 ];
18 sellingPrice = [50 45 80];
19 productionCost = [30 40 55];
20 adjustmentFactor = [1.10 1.05 0.97];
21
22 productNames = ["Product A", "Product B", "Product C"];
23
24 % 1. Size and number of elements of the production matrix
25 [numDays, numProducts] = size(production);
26 totalElements = numel(production);
27
28 fprintf('=== 1. MATRIX PROPERTIES ===\n');
29 fprintf('Size of production matrix : %d rows x %d columns\n', numDays, numProducts);
30 fprintf('Number of working days : %d\n', numDays);
31 fprintf('Number of products : %d\n', numProducts);
32 fprintf('Total number of elements : %d\n', totalElements);
33
34 % 2. Daily and weekly production totals
35 dailyTotals = sum(production, 2); % sum across the columns, one total per day
36 productTotals = sum(production, 1); % sum down the rows, one total per product
37 weeklyTotal = sum(production, 'all');
38
39 fprintf('=== 2. PRODUCTION TOTALS ===\n');
40 for d = 1:numDays
41 fprintf('Day %d total production : %d units\n', d, dailyTotals(d));
42 end
43 fprintf('\n');
44 for p = 1:numProducts
45 fprintf('%s weekly production: %d units\n', productNames(p), productTotals(p));
46 end
47
48 % 3. ORIGINAL WEEKLY FINANCIALS
49
50 % 4. SIMULATED PRODUCTION PLAN
51 Adjustment factors applied: A = 1.10, B = 1.05, C = 0.97
52
53 % 5. PROFIT COMPARISON
54 Original weekly profit : 37600.00
55 Simulated weekly profit : 38770.00
56 Difference : 1170.00
```

```
NAME: PAMESA - AI Declaration ...
PAMESA_hwk1.docx
PAMESA_hwk1.pdf
PAMESA_TechnicalAssessme...

Workspace
Name Value Size Class
adjustmentfa... [1.1000,1.0500,...] 1x3 double
bestids 1 1x1 double
bestProfit 14820 1x1 double
costPerProduct [20190,21000,21... 1x3 double
d 5 1x1 double
dailyTotals [295,311,323,335... 5x1 double
numDays 5 1x1 double
numProducts 3 1x1 double
p 3 1x1 double
percentChange 4.6141 1x1 double
production 5x3 double
productionC_ [30,40,55] 1x3 double
productNames 1x3 string
productTotals [873,525,419] 1x3 double
profitDifferen... 1710 1x1 double
profitPerProd... [13460,13125,10... 1x3 double

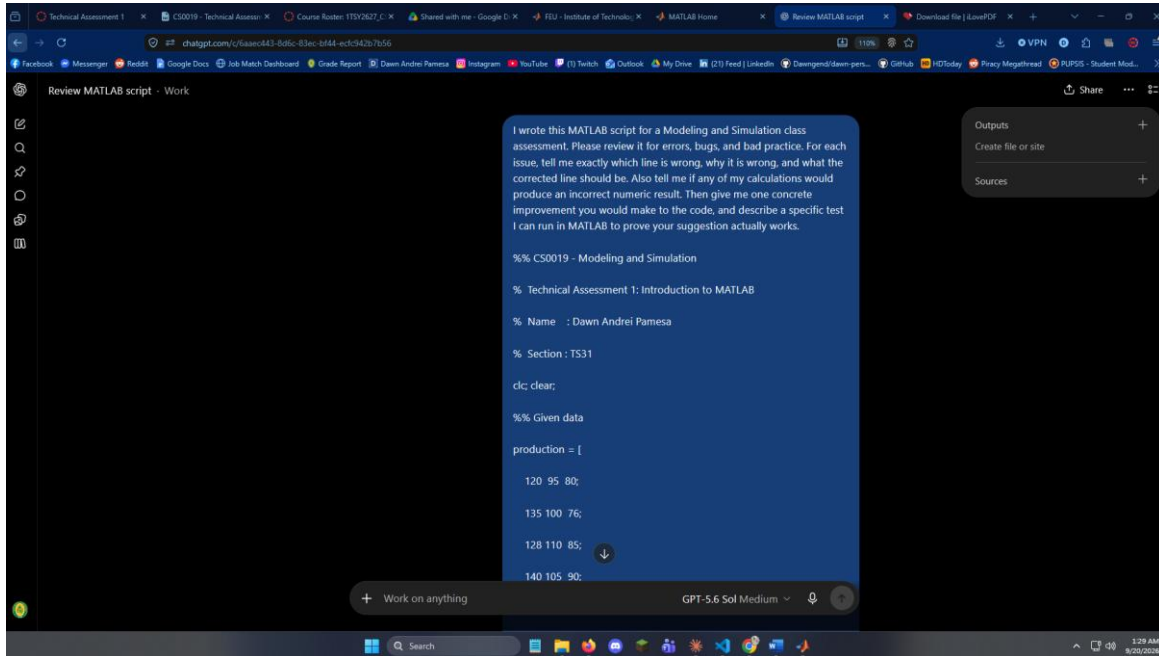
PAMESA_TechnicalAssessment.m
1 % CS0019 - Modeling and Simulation
2 % Technical Assessment 1: Introduction to MATLAB
3 % Name : Dawn Andrei Pamesa
4 % Section : TS31
5
6
7
8 clc; clear;
9
10 % Given data
11 production = [
12 120 95 80;
13 135 100 76;
14 120 110 85;
15 140 105 90;
16 150 115 88
17 ];
18 sellingPrice = [50 45 80];
19 productionCost = [30 40 55];
20 adjustmentFactor = [1.10 1.05 0.97];
21
22 productNames = ["Product A", "Product B", "Product C"];
23
24 % 1. Size and number of elements of the production matrix
25 [numDays, numProducts] = size(production);
26 totalElements = numel(production);
27
28 fprintf('=== 1. MATRIX PROPERTIES ===\n');
29 fprintf('Size of production matrix : %d rows x %d columns\n', numDays, numProducts);
30 fprintf('Number of working days : %d\n', numDays);
31 fprintf('Number of products : %d\n', numProducts);
32 fprintf('Total number of elements : %d\n', totalElements);
33
34 % 2. Daily and weekly production totals
35 dailyTotals = sum(production, 2); % sum across the columns, one total per day
36 productTotals = sum(production, 1); % sum down the rows, one total per product
37 weeklyTotal = sum(production, 'all');
38
39 fprintf('=== 2. PRODUCTION TOTALS ===\n');
40 for d = 1:numDays
41 fprintf('Day %d total production : %d units\n', d, dailyTotals(d));
42 end
43 fprintf('\n');
44 for p = 1:numProducts
45 fprintf('%s weekly production: %d units\n', productNames(p), productTotals(p));
46 end
47
48 % 3. ORIGINAL WEEKLY FINANCIALS
49
50 % 4. SIMULATED PRODUCTION PLAN
51 Adjustment factors applied: A = 1.10, B = 1.05, C = 0.97
52
53 % 5. PROFIT COMPARISON
54 Original weekly profit : 37600.00
55 Simulated weekly profit : 38770.00
56 Difference : 1170.00
57 Percent change : 4.61%
58
59 % 6. RECOMMENDATION
60 ADOPT the proposed production adjustment.
61 It increases weekly profit by 1710.00 (4.61%).
62 Highest contributor under the new plan: Product A (14820.00 profit).
63 >>
```



Paste the snippet of your AI prompts here:

AI tool used: OpenAI ChatGPT

Prompt:



Relevant AI response:

The review reported no calculation errors for the given data set and confirmed every numeric result, including the weekly profit of 37,060, the simulated weekly profit of 38,770, and the 4.61 percent increase. It raised six points, all concerning robustness rather than arithmetic:

1. clear could be clearvars, to state the intent more precisely.
2. The shape of adjustmentFactor is never validated, so a column vector or a wrong element count would fail or expand incorrectly.
3. Rounding is applied per day rather than to weekly totals, which is a modeling assumption, not an error.
4. The simulated output block is hard-coded for exactly three products while the rest of the script uses numProducts.
5. The percentage change divides by weeklyProfit without guarding against zero.
6. max returns only the first product when two products tie for highest profit.

Its single concrete improvement, quoted:

The most valuable improvement is to validate the input dimensions before performing calculations:

```
assert(size(production, 2) == numel(sellingPrice), ...
    'sellingPrice must contain one value per product.');
```

```
assert(size(production, 2) == numel(productionCost), ...
    'productionCost must contain one value per product.');
```

```
assert(size(production, 2) == numel(adjustmentFactor), ...
    'adjustmentFactor must contain one value per product.');
```

```
assert(numel(productNames) == size(production, 2), ...
    'productNames must contain one name per product.');
```

Specific

MATLAB

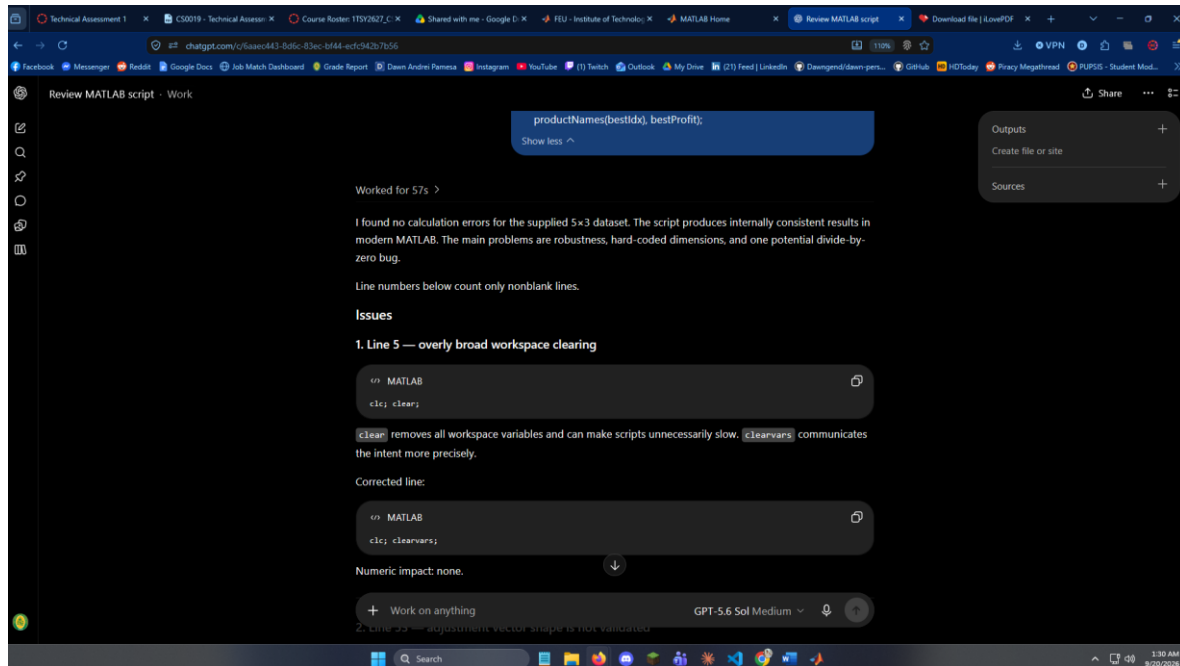
test:



```
adjustmentFactor = [1.10 1.05];  
run('your_script.m');
```

Expected result after adding validation:

There must be exactly one adjustment factor per product.



Documented AI recommendation:

Validate the input dimensions before any calculation, so that a per-product vector with the wrong number of elements is rejected immediately with a message naming the cause, instead of failing later inside an arithmetic operation. This recommendation was adopted and the four assertions now appear in the script directly after the matrix size is determined.

How the recommendation was tested in MATLAB:

Four tests were run in MATLAB R2026a. The recommendation itself was confirmed, but the test procedure the AI supplied to prove it does not work.

Test 1. The AI test procedure exactly as written.

Setting `adjustmentFactor = [1.10 1.05]` and then running the script does not trigger the assertion. The script completed with no error at all and reported "Adjustment factors applied: A = 1.10, B = 1.05, C = 0.97". The reason is line 5 of the script, `clc; clear;`, which empties the workspace before the data block runs, so the two element vector is discarded and the script's own three element vector is used instead. The suggested test can never reach the assertion it is meant to demonstrate, so the procedure was rejected and replaced with the direct tests below.

Test 2. The failure mode without validation.

Evaluating `round(production .* [1.10 1.05])` directly produced error identifier `MATLAB:sizeDimensionsMustMatch` with the message "Arrays have incompatible sizes for this operation." This confirms the AI's claim that the unvalidated script reports a low-level arithmetic error that does not name the real cause.

**Test 3. The same input with the assertion in place.**

The identical two element vector produced the message "There must be exactly one adjustment factor per product." The improvement therefore works as claimed, and reports the actual cause rather than a size mismatch.

Test 4. Regression check on the results.

The complete script was run before and after the four assertions were added, and the two Command Window outputs were compared and found to be identical. Adding the validation therefore changes no reported figure.

A second claim was also checked. The AI stated that rounding the weekly totals instead of each day would give [740 551 406] and a simulated profit of 38,725, an increase of 1,665. Running both interpretations confirmed those figures exactly. The per-day rounding was kept, because the problem describes daily production and a factory cannot produce a fraction of a unit on any given day.

Conclusion: the recommendation was accepted and implemented, its supporting test procedure was rejected as unworkable, and a replacement test was written that demonstrates the improvement directly.



Original and simulated results, interpretation, and recommendation:

A. Original production (units)

Day	Product A	Product B	Product C	Daily Total
Day 1	120	95	80	295
Day 2	135	100	76	311
Day 3	128	110	85	323
Day 4	140	105	90	335
Day 5	150	115	88	353
TOTAL	673	525	419	1617

B. Simulated production (units)

Adjustment factors: $A = 1.10$, $B = 1.05$, $C = 0.97$, each adjusted value rounded to the nearest whole unit.

Day	Product A	Product B	Product C	Daily Total
Day 1	132	100	78	310
Day 2	149	105	74	328
Day 3	141	116	82	339
Day 4	154	110	87	351
Day 5	165	121	85	371
TOTAL	741	552	406	1699

C. Original weekly revenue, cost, and profit

Product	Units	Revenue	Cost	Profit
Product A	673	33,650.00	20,190.00	13,460.00
Product B	525	34,125.00	21,000.00	13,125.00
Product C	419	33,520.00	23,045.00	10,475.00
TOTAL	1,617	101,295.00	64,235.00	37,060.00

D. Simulated weekly revenue, cost, and profit

Product	Units	Revenue	Cost	Profit
Product A	741	37,050.00	22,230.00	14,820.00
Product B	552	35,880.00	22,080.00	13,800.00
Product C	406	32,480.00	22,330.00	10,150.00
TOTAL	1,699	105,410.00	66,640.00	38,770.00

E. Comparison of original and simulated weekly profit

Measure	Original	Simulated	Difference
Weekly production (units)	1,617	1,699	+82
Weekly revenue	101,295.00	105,410.00	+4,115.00
Weekly cost	64,235.00	66,640.00	+2,405.00
Weekly profit	37,060.00	38,770.00	+1,710.00

F. Interpretation

The adjustment factors move output toward Products A and B and away from Product C. Product A gains 68 units for the week and adds 1,360.00 in profit, Product B gains 27 units and adds 675.00, while Product C loses 13 units and gives up 325.00. The two gains outweigh the single loss, so weekly profit rises by 1,710.00, an increase of 4.61 percent.

The gain does not come from producing more units alone. Product C carries the highest production cost of the three at 55.00 per unit, so it ties up the most cost for each unit made, and reducing it is the cheapest cut available. Total production rises by 5.07 percent while profit rises



by 4.61 percent, which shows the improvement comes mainly from shifting the product mix rather than from raising volume.

G. Recommendation

The proposed production adjustment should be adopted. It raises weekly profit from 37,060.00 to 38,770.00 without any change in selling price or unit production cost.

Two conditions should be confirmed before implementation. First, demand for Products A and B must absorb the additional 68 and 27 units, because if the extra output cannot be sold, the added cost of 2,405.00 is incurred without the matching revenue. Second, the 13 unit reduction in Product C must not breach a standing customer commitment, since the model measures profit only and does not account for contractual penalties.

H. Screenshots of the program output

Figure 1. Matrix properties and production totals

Requirements 1 and 2: matrix size, element count, daily totals, weekly totals.

```
Command Window

=== 1. MATRIX PROPERTIES ===
Size of production matrix : 5 rows x 3 columns
Number of working days    : 5
Number of products        : 3
Total number of elements  : 15

=== 2. PRODUCTION TOTALS ===
Day 1 total production    : 295 units
Day 2 total production    : 311 units
Day 3 total production    : 323 units
Day 4 total production    : 335 units
Day 5 total production    : 353 units

Product A weekly production: 673 units
Product B weekly production: 525 units
Product C weekly production: 419 units
Total weekly production    : 1617 units
```

Figure 2. Original weekly revenue, cost, and profit

=== 3. ORIGINAL WEEKLY FINANCIALS ===			
Product	Revenue	Cost	Profit
Product A	33650.00	20190.00	13460.00
Product B	34125.00	21000.00	13125.00
Product C	33520.00	23045.00	10475.00
TOTAL	101295.00	64235.00	37060.00

Requirement 3: weekly financial results for the original plan.



Figure 3. Simulated production plan and financials

```
=== 4. SIMULATED PRODUCTION PLAN ===  
Adjustment factors applied: A = 1.10, B = 1.05, C = 0.97  
  
Day      Product A    Product B    Product C    Day Total  
Day 1      132         100         78         310  
Day 2      149         105         74         328  
Day 3      141         116         82         339  
Day 4      154         110         87         351  
Day 5      165         121         85         371  
TOTAL      741         552         406        1699  
  
=== SIMULATED WEEKLY FINANCIALS ===  
Product      Revenue      Cost      Profit  
Product A    37050.00    22230.00    14820.00  
Product B    35880.00    22080.00    13800.00  
Product C    32480.00    22330.00    10150.00  
TOTAL        105410.00    66640.00    38770.00
```

Requirement 4: production after the adjustment factors, with revenue, cost, profit.

Figure 4. Profit comparison and recommendation

```
Command Window  
  
=== 5. PROFIT COMPARISON ===  
Original weekly profit : 37060.00  
Simulated weekly profit : 38770.00  
Difference : 1710.00  
Percent change : 4.61%  
  
=== 6. RECOMMENDATION ===  
ADOPT the proposed production adjustment.  
It increases weekly profit by 1710.00 (4.61%).  
Highest contributor under the new plan: Product A (14820.00 profit).  
>> |
```

Requirements 5 and 6: profit comparison and the recommendation from the script.